

If you just wanted a list of the keys ('the', 'a', 'ophthalmologist') of a dict, you can use the method keys as: 'dictvar.keys()'

Likewise to get the values you can use: 'dictvar.values()'

You can also use 'dictvar.items()' to get a list of tuple containing key-value pairs.

Sets

Sets are once again very similar to lists, but with the restriction that each value may only be added to it once. It can be quite useful for getting the unique values in a list. To convert an existing list to a set, you can use simply pass it to the set constructor method as a parameter. So `set([1,2,2,3,4])` will return `{1,2,3,4}`

Sets can also be incredibly useful in finding intersections, unions and differences between multiple lists.

To create a new empty set use the following:

- `s = set()`

You can now add elements to this set using `s.add(value)`.

Functions

Functions in Python are similar to most other languages, but support one important feature that makes them quite pleasant to use. That is their ability to specify parameter names. Let's look at a simple function that prints a string a specified number of times:

- `def repeat_print(string, count=1):`
- `for _ in range(count):`
- `print(string)`

There are a lot of interesting things going on here, first of all this function supports two parameters, 'string' and 'count'. The default value of 'count' is set to 1 so this parameter can be omitted to print a string once.

Second of all we are using the underscore as a variable name in the for loop. This is common practice when we don't care about the value of the variable and will not be accessing it. Here we just need the loop to run a set number of times. You'll also notice that if we just provide a single number inside the range function, it will simply count to that number from zero which works for us here.

We can now easily call this function as `repeat_print("string to repeat", 5)`, but other interesting ways to do the same are as follows:

- `repeat_print("hi", count=4)`